

☒ Nirvigh Advisors HRMS — Full Production-Level AI Development Prompt

Company: Nirvigh Advisors (Real Estate)

Type: Full-Stack Web Application (HRMS — Human Resource Management System)

Roles: HR Admin (controller), Employee (self-service)

Authentication: Invite-only employee registration (no public sign-up)

Goal: Build a production-grade, secure, role-based HRMS web app from scratch, divided into clear phases.

☒ ARCHITECTURE OVERVIEW (Read Before Everything Else)

Build this as a full-stack web application with the following structure:

```
nirvigh-hrms/
├── frontend/                                ← React (Vite) or Next.js 14 App Router
│   ├── src/
│   │   ├── app/ or pages/
│   │   ├── components/
│   │   ├── features/                        ← feature-based folder structure
│   │   │   ├── auth/
│   │   │   ├── employees/
│   │   │   ├── attendance/
│   │   │   ├── leave/
│   │   │   ├── payroll/
│   │   │   ├── expenses/
│   │   │   ├── performance/
│   │   │   ├── documents/
│   │   │   └── reports/
│   │   ├── hooks/
│   │   ├── utils/
│   │   ├── store/                          ← Zustand or Redux Toolkit
│   │   └── styles/
│   └── public/
├── backend/                                ← Node.js + Express.js (or NestJS)
│   ├── src/
│   │   ├── modules/
│   │   │   ├── auth/
│   │   │   ├── users/
│   │   │   ├── employees/
│   │   │   ├── attendance/
│   │   │   ├── leave/
│   │   │   ├── payroll/
│   │   │   ├── expenses/
│   │   │   ├── performance/
│   │   │   ├── documents/
│   │   │   └── reports/
```

```

|   |   |   | middleware/
|   |   |   |   | auth.middleware.js      ← JWT verification
|   |   |   |   | rbac.middleware.js      ← Permission checks
|   |   |   |   | audit.middleware.js     ← Action logging
|   |   |   |   | upload.middleware.js    ← File uploads (Multer)
|   |   |   | config/
|   |   |   | utils/
|   |   |   | app.js
|   |   |   | prisma/ or db/
|   |   |   |   | schema.prisma           ← Database schema
|   |   |   |
|   |   |   | docker-compose.yml        ← PostgreSQL + Redis + App containers

```

Tech Stack (exact versions):

- **Frontend:** React 18 + Vite, React Router v6, Zustand (state), Axios (HTTP), React Query (server state), Tailwind CSS v3, shadcn/ui or Radix UI components, Recharts (data viz)
- **Backend:** Node.js v20+, Express.js v4, Prisma ORM (PostgreSQL), JWT (jsonwebtoken), bcryptjs, Nodemailer, Multer (file uploads), Zod (validation), Winston (logging)
- **Database:** PostgreSQL 15
- **Cache:** Redis (for sessions, rate limiting, invite tokens)
- **Storage:** AWS S3 or Cloudflare R2 (document/file uploads) — OR local disk in dev
- **Email:** Nodemailer + SMTP (Gmail/Mailgun/SendGrid for production)
- **Deployment:** Docker + Docker Compose. Frontend on Vercel or Netlify. Backend on Railway, Render, or DigitalOcean. DB on Supabase or managed PostgreSQL.

☒ HOW EMPLOYEE REGISTRATION WORKS (Most Important — Read First)

There is NO public sign-up page. Employees cannot register themselves directly. Here is the exact invite-based flow:

Step 1 — HR Admin Creates Employee Profile

HR Admin logs in → goes to "Add Employee" → fills in:

- Full name, work email, designation, department, work site, date of joining, employment type

System creates:

- Employee record in DB (status: PENDING_INVITE)
- User record linked to employee (status: INVITED, role: EMPLOYEE)
- Generates a secure random 64-byte invite token
- Stores token hash in DB with 72-hour expiry
- Sends an invite email to the work email containing the link:
<https://hrms.nirvighadvisors.com/activate?token=<TOKEN>>

Step 2 — Employee Activates Account

Employee clicks link → lands on Activate Account page:

- Verify token is valid and not expired
- Form: Set Password (min 8 chars, 1 uppercase, 1 number, 1 special char) + Confirm Password
- On submit: hash password, save to User, mark User status as ACTIVE, delete/invalidate invite token
- Auto-login and redirect to Employee Onboarding flow

Step 3 — Employee Completes Onboarding

After activation, show a 4-step onboarding wizard:

1. **Personal Info:** DOB, gender, personal email, phone, alternate phone
2. **Address:** Current address, permanent address (checkbox: same as current)
3. **Emergency Contact:** Name, relationship, phone
4. **Documents Upload:** PAN card image, Aadhaar card image, bank passbook/canceled cheque, profile photo

Onboarding can be skipped and completed later from "My Profile". HR Admin can see which employees have incomplete profiles.

Step 4 — Subsequent Logins

Employee visits `/login` → enters work email + password → JWT issued → redirected to Employee Dashboard.

Token Security Rules:

- Invite tokens are single-use (invalidated immediately after activation)
- Tokens expire in 72 hours
- If expired: HR Admin can resend invite from the employee's profile page
- Token is stored as SHA-256 hash in DB; the raw token is only in the email
- After 5 failed login attempts: account locked for 30 minutes, admin notified

📦 DATABASE SCHEMA (Prisma / PostgreSQL)

Build these exact models:

User Model

```
model User {
  id          String    @id @default(uuid())
  email       String    @unique
  passwordHash String?
  role        Role      @default(EMPLOYEE)
  status      UserStatus @default(INVITED)
  inviteToken String?   @unique
  inviteExpiresAt DateTime?
  lastLoginAt DateTime?
  loginAttempts Int      @default(0)
  lockedUntil  DateTime?
  createdAt    DateTime  @default(now())
  updatedAt    DateTime  @updatedAt
  employee     Employee?
  auditLogs    AuditLog[]
}

enum Role { HR_ADMIN EMPLOYEE }
enum UserStatus { INVITED ACTIVE INACTIVE LOCKED }
```

Employee Model

```
model Employee {
  id          String    @id @default(uuid())
  employeeCode String    @unique // e.g. NVA-0042
  userId      String    @unique
  user        User      @relation(fields: [userId], references: [id])

  // Personal
  firstName   String
  lastName    String
  gender      Gender?
  dateOfBirth DateTime?
  profilePhotoUrl String?
  personalEmail String?
  phone       String?
  alternatePhone String?
  currentAddress String?
  permanentAddress String?
  emergencyContactName String?
  emergencyContactRel String?
  emergencyContactPhone String?

  // Employment
  dateOfJoining DateTime
  dateOfLeaving DateTime?
  employmentType EmploymentType @default(FULL_TIME)
  designation    String
  departmentId    String
  department      Department @relation(fields: [departmentId], references: [id])
  workSiteId      String?
  workSite        WorkSite? @relation(fields: [workSiteId], references: [id])
}
```

```

managerId      String?
manager        Employee? @relation("Reports", fields: [managerId], reference
reportees      Employee[] @relation("Reports")
status         EmployeeStatus @default(ACTIVE)

// Sensitive (encrypted at app level)
panNumber      String?
aadhaarNumber  String?
bankAccountNumber String?
bankIFSC       String?
bankName       String?

// Relations
salaryStructureId String?
salaryStructure  SalaryStructure? @relation(fields: [salaryStructureId], refere
attendance       AttendanceRecord[]
leaveBalances    LeaveBalance[]
leaveRequests    LeaveRequest[]
payslips         Payslip[]
expenses         ExpenseClaim[]
documents        EmployeeDocument[]
goalsAssigned    Goal[]
reviews          Review[]
createdAt        DateTime @default(now())
updatedAt        DateTime @updatedAt
}

enum Gender { MALE FEMALE OTHER PREFER_NOT_TO_SAY }
enum EmploymentType { FULL_TIME PART_TIME CONTRACTOR INTERN }
enum EmployeeStatus { ACTIVE INACTIVE TERMINATED ON_LEAVE }

```

Department and WorkSite

```

model Department {
  id      String    @id @default(uuid())
  name    String    @unique // e.g. Sales, Operations, Finance, HR, Site Manage
  employees Employee[]
  createdAt DateTime @default(now())
}

model WorkSite {
  id      String    @id @default(uuid())
  name    String    // e.g. "Thane Project", "Head Office Vira
  address String?
  latitude Float?
  longitude Float?
  radius  Int?      @default(200) // geofencing radius in meters
  employees Employee[]
  attendance AttendanceRecord[]
  createdAt DateTime @default(now())
}

```

Attendance

```
model AttendanceRecord {
  id          String    @id @default(uuid())
  employeeId  String
  employee    Employee  @relation(fields: [employeeId], references: [id])
  date        DateTime  @db.Date
  punchInTime DateTime?
  punchOutTime DateTime?
  punchInLat  Float?
  punchInLng  Float?
  punchOutLat Float?
  punchOutLng Float?
  workSiteId  String?
  workSite    WorkSite? @relation(fields: [workSiteId], references: [id])
  source      AttendanceSource @default(WEB)
  status      AttendanceStatus
  workHours   Float?          // computed on punch-out
  isRegularized Boolean @default(false)
  regularizationReq RegularizationRequest?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@unique([employeeId, date])
}

model RegularizationRequest {
  id          String    @id @default(uuid())
  attendanceId String    @unique
  attendance  AttendanceRecord @relation(fields: [attendanceId], references: [id])
  reason      String
  requestedIn  DateTime?
  requestedOut DateTime?
  status      ApprovalStatus @default(PENDING)
  reviewedById String?
  reviewedAt  DateTime?
  remarks     String?
  createdAt   DateTime @default(now())
}

enum AttendanceSource { WEB MOBILE GPS_VERIFIED }
enum AttendanceStatus { PRESENT ABSENT HALF_DAY LATE WFH ON_LEAVE HOLIDAY WEEKEND }
enum ApprovalStatus { PENDING APPROVED REJECTED CANCELLED }
```

Leave

```
model LeaveType {
  id          String    @id @default(uuid())
  name        String    @unique // CL, SL, PL, LWP, Comp-Off
  code        String    @unique
  description  String?
  isPaid      Boolean    @default(true)
  maxDaysPerYear Int     @default(12)
  carryForward Boolean    @default(false)
```

```

    maxCarryForward Int?
    requiresDoc      Boolean @default(false) // SL over 2 days needs medical cert
    noticeDays       Int      @default(0)
    balances          LeaveBalance[]
    requests          LeaveRequest[]
    createdAt         DateTime @default(now())
}

model LeaveBalance {
  id          String @id @default(uuid())
  employeeId  String
  employee    Employee @relation(fields: [employeeId], references: [id])
  leaveTypeId String
  leaveType   LeaveType @relation(fields: [leaveTypeId], references: [id])
  year        Int
  totalDays   Float
  usedDays    Float @default(0)
  pendingDays Float @default(0)
  remainingDays Float // computed: totalDays - usedDays - pendingDays

  @@unique([employeeId, leaveTypeId, year])
}

model LeaveRequest {
  id          String @id @default(uuid())
  employeeId  String
  employee    Employee @relation(fields: [employeeId], references: [id])
  leaveTypeId String
  leaveType   LeaveType @relation(fields: [leaveTypeId], references: [id])
  startDate   DateTime @db.Date
  endDate     DateTime @db.Date
  totalDays   Float
  reason       String
  attachment  String? // URL for medical cert etc.
  status       ApprovalStatus @default(PENDING)
  approvedById String?
  approvedAt   DateTime?
  remarks      String?
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
}

```

Payroll

```

model SalaryStructure {
  id          String @id @default(uuid())
  name        String @unique // e.g. "L1 Sales Staff", "L3 Manager"
  components  SalaryComponent[]
  employees   Employee[]
  createdAt   DateTime @default(now())
}

model SalaryComponent {
  id          String @id @default(uuid())
  structureId String

```

```

    structure      SalaryStructure @relation(fields: [structureId], references: [id]
    name           String          // Basic, HRA, Conveyance, Medical, Gross, PF, Professi
    type           ComponentType    // EARNING, DEDUCTION
    calculationType CalcType        // FIXED, PERCENTAGE_OF_BASIC, PERCENTAGE_OF_GROSS
    value          Float            // the fixed amount or percentage
    order          Int              // display order
}

model PayrollRun {
  id           String    @id @default(uuid())
  month        Int
  year         Int
  status       PayrollStatus @default(DRAFT)
  generatedById String
  processedAt  DateTime?
  lockedAt     DateTime?
  payslips    Payslip[]
  createdAt   DateTime  @default(now())

  @@unique([month, year])
}

model Payslip {
  id           String    @id @default(uuid())
  employeeId   String
  employee     Employee  @relation(fields: [employeeId], references: [id])
  payrollRunId String
  payrollRun   PayrollRun @relation(fields: [payrollRunId], references: [id])
  month        Int
  year         Int
  basicSalary  Float
  earnings     Json       // { "HRA": 5000, "Conveyance": 1600, ... }
  deductions   Json       // { "PF": 1800, "PT": 200, "TDS": 0, ... }
  grossEarnings Float
  totalDeductions Float
  netPayable   Float
  workingDays  Int
  presentDays  Float
  pdfUrl       String?
  generatedAt  DateTime  @default(now())
}

enum ComponentType { EARNING DEDUCTION }
enum CalcType { FIXED PERCENTAGE_OF_BASIC PERCENTAGE_OF_GROSS }
enum PayrollStatus { DRAFT PROCESSING COMPLETED LOCKED }

```

Expenses

```

model ExpenseClaim {
  id           String    @id @default(uuid())
  employeeId   String
  employee     Employee  @relation(fields: [employeeId], references: [id])
  title        String
  category     ExpenseCategory
  totalAmount  Float
}

```



```

    items      ExpenseItem[]
    status      ApprovalStatus @default(PENDING)
    approvedById String?
    approvedAt   DateTime?
    remarks      String?
    paymentStatus PaymentStatus @default(UNPAID)
    paidAt       DateTime?
    createdAt    DateTime @default(now())
    updatedAt    DateTime @updatedAt
}

model ExpenseItem {
  id          String    @id @default(uuid())
  claimId     String
  claim       ExpenseClaim @relation(fields: [claimId], references: [id])
  description String
  amount      Float
  date        DateTime @db.Date
  receiptUrl  String?
}

enum ExpenseCategory { TRAVEL LOCAL_TRANSPORT ACCOMMODATION MEALS CLIENT_ENTERTAINME
enum PaymentStatus { UNPAID APPROVED_FOR_PAYMENT PAID }

```

Performance

```

model ReviewCycle {
  id          String    @id @default(uuid())
  name        String    // e.g. "H1 2025 Review"
  startDate   DateTime
  endDate     DateTime
  status      CycleStatus @default(ACTIVE)
  goals       Goal[]
  reviews     Review[]
  createdAt   DateTime @default(now())
}

model Goal {
  id          String    @id @default(uuid())
  employeeId  String
  employee    Employee @relation(fields: [employeeId], references: [id])
  cycleId     String
  cycle       ReviewCycle @relation(fields: [cycleId], references: [id])
  title       String
  description  String?
  targetDate  DateTime?
  status      GoalStatus @default(NOT_STARTED)
  progress    Int        @default(0) // 0-100
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model Review {
  id          String    @id @default(uuid())
  employeeId  String

```

```

employee      Employee @relation(fields: [employeeId], references: [id])
cycleId       String
cycle         ReviewCycle @relation(fields: [cycleId], references: [id])
selfRating    Float?    // 1-5
selfComments  String?
hrRating      Float?
hrComments    String?
finalRating   Float?
status        ReviewStatus @default(SELF_REVIEW_PENDING)
submittedAt   DateTime?
completedAt   DateTime?
createdAt     DateTime @default(now())
updatedAt     DateTime @updatedAt
}

enum CycleStatus { ACTIVE COMPLETED ARCHIVED }
enum GoalStatus { NOT_STARTED IN_PROGRESS COMPLETED CANCELLED }
enum ReviewStatus { SELF_REVIEW_PENDING SELF_SUBMITTED HR_REVIEW_PENDING COMPLETED }

```

Documents and Audit

```

model EmployeeDocument {
  id          String    @id @default(uuid())
  employeeId  String
  employee    Employee @relation(fields: [employeeId], references: [id])
  type        DocumentType
  fileName    String
  fileUrl     String
  fileSize    Int?
  uploadedById String
  expiresAt   DateTime?
  createdAt   DateTime @default(now())
}

model PolicyDocument {
  id          String    @id @default(uuid())
  title       String
  description  String?
  fileUrl     String
  version     String    @default("1.0")
  isActive    Boolean   @default(true)
  publishedAt DateTime @default(now())
  createdAt   DateTime @default(now())
}

model AuditLog {
  id          String    @id @default(uuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  action      String    // e.g. "EMPLOYEE_UPDATED", "PAYROLL_RUN", "LEAVE_APPROVED"
  targetId    String?   // ID of the affected resource
  targetType  String?   // "Employee", "LeaveRequest", etc.
  details     Json?     // diff or extra details
  ipAddress   String?
  userAgent   String?
}

```

```

    createdAt    DateTime    @default(now())
  }

  model Notification {
    id            String      @id @default(uuid())
    userId        String
    title         String
    message       String
    type          NotifType
    isRead        Boolean     @default(false)
    linkUrl       String?
    createdAt     DateTime    @default(now())
  }

  enum DocumentType { PAN_CARD AADHAAR BANK_PASSBOOK PROFILE_PHOTO OFFER_LETTER CONTRA
  enum NotifType { LEAVE ATTENDANCE PAYROLL EXPENSE PERFORMANCE SYSTEM }

```

☒ RBAC PERMISSION SYSTEM

Create a `permissions.js` config file:

```

export const PERMISSIONS = {
  // Employee Management
  EMPLOYEE_VIEW_ALL: 'employee:view_all',
  EMPLOYEE_VIEW_SELF: 'employee:view_self',
  EMPLOYEE_CREATE: 'employee:create',
  EMPLOYEE_EDIT_ALL: 'employee:edit_all',
  EMPLOYEE_EDIT_SELF_PARTIAL: 'employee:edit_self_partial',
  EMPLOYEE_TERMINATE: 'employee:terminate',
  EMPLOYEE_INVITE: 'employee:invite',

  // Attendance
  ATTENDANCE_VIEW_ALL: 'attendance:view_all',
  ATTENDANCE_VIEW_SELF: 'attendance:view_self',
  ATTENDANCE_APPROVE: 'attendance:approve',
  ATTENDANCE_CONFIGURE: 'attendance:configure',

  // Leave
  LEAVE_VIEW_ALL: 'leave:view_all',
  LEAVE_VIEW_SELF: 'leave:view_self',
  LEAVE_APPLY: 'leave:apply',
  LEAVE_APPROVE: 'leave:approve',
  LEAVE_CONFIGURE: 'leave:configure',

  // Payroll
  PAYROLL_VIEW_ALL: 'payroll:view_all',
  PAYROLL_VIEW_SELF: 'payroll:view_self',
  PAYROLL_RUN: 'payroll:run',
  PAYROLL_CONFIGURE: 'payroll:configure',

  // Expenses
  EXPENSE_VIEW_ALL: 'expense:view_all',
  EXPENSE_SUBMIT: 'expense:submit',
  EXPENSE_APPROVE: 'expense:approve',

```

```

// Performance
PERFORMANCE_VIEW_ALL: 'performance:view_all',
PERFORMANCE_SELF_REVIEW: 'performance:self_review',
PERFORMANCE_MANAGE: 'performance:manage',

// Documents
DOCUMENT_VIEW_ALL: 'document:view_all',
DOCUMENT_VIEW_SELF: 'document:view_self',
DOCUMENT_UPLOAD_FOR_EMPLOYEE: 'document:upload_for_employee',
DOCUMENT_UPLOAD_SELF: 'document:upload_self',
POLICY_MANAGE: 'document:policy_manage',

// Reports
REPORTS_FULL: 'reports:full',

// Settings
SETTINGS_MANAGE: 'settings:manage',
}

export const ROLE_PERMISSIONS = {
  HR_ADMIN: Object.values(PERMISSIONS), // all permissions
  EMPLOYEE: [
    PERMISSIONS.EMPLOYEE_VIEW_SELF,
    PERMISSIONS.EMPLOYEE_EDIT_SELF_PARTIAL,
    PERMISSIONS.ATTENDANCE_VIEW_SELF,
    PERMISSIONS.LEAVE_VIEW_SELF,
    PERMISSIONS.LEAVE_APPLY,
    PERMISSIONS.PAYROLL_VIEW_SELF,
    PERMISSIONS.EXPENSE_SUBMIT,
    PERMISSIONS.PERFORMANCE_SELF_REVIEW,
    PERMISSIONS.DOCUMENT_VIEW_SELF,
    PERMISSIONS.DOCUMENT_UPLOAD_SELF,
  ]
}

```

Backend middleware:

```

// rbac.middleware.js
export const requirePermission = (permission) => (req, res, next) => {
  const userPermissions = ROLE_PERMISSIONS[req.user.role] || []
  if (!userPermissions.includes(permission)) {
    return res.status(403).json({ error: 'Insufficient permissions' })
  }
  next()
}

```

☒ DESIGN SYSTEM

Art Direction: Professional, clean, trustworthy. Real estate company — think structured and authoritative with a hint of warmth.

Color Palette:

- Primary: Deep Teal #01696f (hover: #0c4e54, active: #0f3638)
- Background (light): #f7f6f2 (warm off-white)
- Background (dark): #171614
- Surface: #f9f8f5 / dark: #1c1b19
- Accent for success: #437a22
- Error: #a12c7b
- Warning: #964219
- Text: #28251d / dark: #cdccca
- Text muted: #7a7974 / dark: #797876

Typography:

- Display/Headings: 'Satoshi' from Fontshare (weights 500, 700)
- Body/UI: 'General Sans' from Fontshare (weights 400, 500, 600)

Type Scale (Web App — compact):

- Page title: $\text{clamp}(1.5\text{rem}, 1.2\text{rem} + 1.25\text{vw}, 2.25\text{rem}) \rightarrow \text{max } 36\text{px}$
- Section headings: $\text{clamp}(1.125\text{rem}, 1\text{rem} + 0.75\text{vw}, 1.5\text{rem}) \rightarrow 18\text{--}24\text{px}$
- Body: $\text{clamp}(1\text{rem}, 0.95\text{rem} + 0.25\text{vw}, 1.125\text{rem}) \rightarrow 16\text{--}18\text{px}$
- Buttons/Nav: 14px
- Labels/badges: 12px floor

Component Design Rules:

- Border radius: inputs/buttons 0.5rem, cards 0.75rem, modals 1rem
- All borders: 1px solid $\text{oklch}(\text{from var(--color-text)} \text{ l c h } / 0.12)$ — never solid gray
- Shadows: 0 1px 2px $\text{oklch}(0.2 \ 0.01 \ 80 / 0.06)$, 0 4px 12px $\text{oklch}(0.2 \ 0.01 \ 80 / 0.08)$ for cards
- No colored side borders on cards (classic AI-look anti-pattern)
- No gradient buttons — solid teal only
- No icons in colored circles for features
- Sidebar navigation (persistent, collapsible on mobile)
- Top bar: notifications bell, user avatar dropdown

☒ PHASE BREAKDOWN

✓ PHASE 1 — DESIGN SYSTEM + UI FOUNDATIONS

Goal: Establish the entire design language and reusable component library before writing any business logic.

1A. Design Tokens (style.css)

Create a single `tokens.css` or Tailwind config with:

- All color variables (light + dark mode via `data-theme`)
- Fluid type scale with `clamp()` for all 6 size tokens
- 4px spacing system (space-1 through space-32)
- Border radius tokens (sm, md, lg, xl, full)
- Shadow tokens
- Transition: `180ms cubic-bezier(0.16, 1, 0.3, 1)` for interactive states
- Font families (load Satoshi + General Sans from Fontshare CDN)

1B. Base Component Library

Build these reusable components (React functional components with TypeScript):

Atoms:

- `<Button>` — variants: primary, secondary, ghost, danger; sizes: sm, md, lg; loading state with spinner
- `<Input>` — label, error state, helper text, required indicator, password show/hide toggle
- `<Textarea>` — same as Input
- `<Select>` — custom styled, searchable

```
- `<Checkbox>` + `<RadioGroup>`
```

- `<Badge>` — variants: success, warning, error, neutral, info; sizes: sm, md
- `<Avatar>` — image with initials fallback
- `<Spinner>` — 3 sizes
- `<Skeleton>` — shimmer animation, variants: text, heading, avatar, image, card

Molecules:

- `<Card>` — surface-elevated, optional header/footer
- `<Modal>` — with overlay, title, close, scrollable body, footer actions
- `<Drawer>` — slide-in from right, for forms/details

```
- `<Toast>` / `<Notification>` — stacked, auto-dismiss, 4 variants
```

- `<ConfirmDialog>` — for destructive actions
- `<Tabs>` — horizontal, with badge counts
- `<DataTable>` — sortable columns, pagination, row selection, loading skeleton, empty state, search input
- `<FormGroup>` — label + input + error wrapper
- `<StatsCard>` — KPI widget: icon, value, label, delta (up/down %), trend sparkline
- `<PageHeader>` — title, breadcrumbs, action button slot
- `<EmptyState>` — icon, title, description, optional CTA button
- `<FileUpload>` — drag-drop zone, preview, progress bar, allowed types

Layout:

- `<AppShell>` — sidebar + topbar + main content wrapper
- `<Sidebar>` — nav items, role-aware (shows only permitted links), collapsible
- `<TopBar>` — logo, page title, search (cmd+K), notification bell with unread count, user avatar dropdown

1C. Route Structure (React Router v6)

```

/login                → LoginPage
/activate             → ActivateAccountPage (token query param)
/forgot-password      → ForgotPasswordPage
/reset-password       → ResetPasswordPage

/admin/*              → HR Admin routes (guard: HR_ADMIN only)
  /admin/dashboard
  /admin/employees
  /admin/employees/new
  /admin/employees/:id
  /admin/employees/:id/edit
  /admin/employees/:id/documents
  /admin/attendance
  /admin/attendance/:employeeId
  /admin/leave
  /admin/leave/types
  /admin/leave/balances
  /admin/payroll
  /admin/payroll/structures
  /admin/payroll/run
  /admin/payroll/:runId
  /admin/expenses
  /admin/performance
  /admin/performance/cycles
  /admin/reports
  /admin/documents/policies
  /admin/settings

/employee/*           → Employee routes (guard: EMPLOYEE only)
  /employee/dashboard
  /employee/profile

```

```
/employee/profile/edit
/employee/attendance
/employee/leave
/employee/leave/apply
/employee/payslips
/employee/expenses
/employee/expenses/new
/employee/performance
/employee/documents
/employee/notifications
```

1D. Auth & Route Guards

- `<ProtectedRoute>` component: checks JWT in memory, verifies role, redirects to `/login` if not authenticated
- `<AuthLayout>` for login/activation pages (centered card, logo, no sidebar)
- `<AppLayout>` for all authenticated pages (sidebar + topbar)
- Store auth state in Zustand: `{ user, token, isAuthenticated, login(), logout() }`
- JWT stored in `httpOnly` cookie (NOT `localStorage`) — set by backend on login response
- Axios interceptor: attach JWT from cookie automatically; on 401 response, clear auth state and redirect to login

✓ PHASE 2 — AUTH + EMPLOYEE MANAGEMENT (Backend + Frontend)

Goal: Full authentication system, invite flow, and employee CRUD.

2A. Backend — Auth Module

Endpoints:

POST	<code>/api/auth/login</code>	→ <code>{ email, password } → { user, token }</code>
POST	<code>/api/auth/logout</code>	→ clears cookie
POST	<code>/api/auth/activate</code>	→ <code>{ token, password } → activates account</code>
POST	<code>/api/auth/forgot-password</code>	→ <code>{ email } → sends reset link</code>
POST	<code>/api/auth/reset-password</code>	→ <code>{ token, newPassword }</code>
GET	<code>/api/auth/me</code>	→ returns current user + employee profile

Login logic:

1. Find user by email
2. Check status (reject `INVITED/LOCKED/INACTIVE` with specific message)
3. Check `lockedUntil` — if future timestamp, reject with lockout message
4. Verify `bcrypt` password
5. On failure: increment `loginAttempts`; if ≥ 5 , set `lockedUntil = now + 30min`
6. On success: reset `loginAttempts`, set `lastLoginAt`, return JWT in `httpOnly` cookie + user data in JSON

7. Log to AuditLog

JWT payload: { `userId`, `role`, `employeeId` }

2B. Backend — Employee Module

Endpoints:

```
GET    /api/employees           → list (admin: all; employee: only self via /api/employees/:id)
POST   /api/employees           → create employee + user + send invite [HR_ADMIN]
GET    /api/employees/:id      → get employee profile
PATCH /api/employees/:id      → update employee [HR_ADMIN: all fields; EMPLOYEE: own fields]
DELETE /api/employees/:id      → soft delete (mark TERMINATED) [HR_ADMIN]
POST   /api/employees/:id/resend-invite → resend activation email [HR_ADMIN]
GET    /api/employees/:id/documents → get employee documents
POST   /api/employees/:id/documents → upload document
```

Create Employee logic (backend):

1. Validate request body with Zod schema
2. Check email uniqueness
3. Generate unique `employeeCode`: NVA- + padded 4-digit increment (e.g., NVA-0042)
4. Create Employee record and linked User record in a DB transaction
5. Generate 64-byte random token (`crypto.randomBytes`), store SHA-256 hash in DB with `inviteExpiresAt = now + 72h`
6. Send invite email via Nodemailer with the raw token in the link
7. Create AuditLog entry
8. Return created employee (without sensitive fields)

Employee update — partial self-edit allowed fields:

`personalEmail`, `phone`, `alternatePhone`, `currentAddress`, `permanentAddress`,
`emergencyContactName`, `emergencyContactRel`, `emergencyContactPhone`

All other fields (designation, department, salary, etc.) editable by HR_ADMIN only.

2C. Frontend — Employee Management Pages

HR Admin — Employee List Page:

- Full-width DataTable
- Columns: Avatar+Name, Employee Code, Designation, Department, Work Site, Status (badge), Date of Joining, Actions
- Filters bar: search by name/code/email, filter by Department, Status, Employment Type
- "Add Employee" button → opens <Drawer> (not modal) with Add Employee form
- Row click → navigate to employee detail page
- Bulk actions: export to CSV, bulk deactivate

HR Admin — Add Employee Form (in Drawer):

Fields (all validated):

- First Name *, Last Name *
- Work Email * (unique check on blur)
- Designation *, Department * (dropdown), Work Site (dropdown)
- Employment Type * (Full-time / Part-time / Contractor / Intern)
- Date of Joining * (date picker)
- Reporting Manager (employee search dropdown, optional)
- Basic Salary (optional at creation, can add later)

On submit: create employee + send invite. Show success toast: "Employee created. Invite sent to [email]."

HR Admin — Employee Profile Page:

Tabbed layout (tabs: Overview, Attendance, Leave, Payroll, Documents, Performance):

- **Overview tab:** Two-column layout. Left: profile photo + key info card (code, designation, department, status badge, joined date). Right: tabs for Personal Info, Employment Info, Bank Details (HR_ADMIN only), Emergency Contact.
- Each info section has an edit button that opens an inline edit form or drawer.
- "Resend Invite" button shown if user status is INVITED and invite expired.
- "Terminate Employee" button with confirm dialog.

2D. Frontend — Employee Onboarding Wizard

4-step wizard shown after first login (if onboarding incomplete):

- Step indicator at top (1 → 2 → 3 → 4 with labels)
- Back/Next/Skip later navigation
- Step 1: Personal Info form (DOB, gender, personal email, phone)
- Step 2: Address form (current + permanent with "same as current" checkbox)
- Step 3: Emergency Contact form
- Step 4: Document upload (drag-drop zones for each document type)
- "Complete Later" skips to dashboard but shows a banner: "Your profile is incomplete. [Complete now →]"

✓ **PHASE 3 — ATTENDANCE & LEAVE (Backend + Frontend)**

Goal: Complete attendance tracking and leave management system.

3A. Backend — Attendance Module

Endpoints:

GET	/api/attendance	→ all records [HR_ADMIN]; own records [EMPLOYEE]
GET	/api/attendance/today	→ today's status for current user
POST	/api/attendance/punch-in	→ { latitude, longitude } → create/update today's
POST	/api/attendance/punch-out	→ { latitude, longitude } → update punchOutTime,
GET	/api/attendance/:employeeId	→ employee's attendance calendar [HR_ADMIN]
PATCH	/api/attendance/:id	→ manual correction [HR_ADMIN]
GET	/api/attendance/summary	→ monthly summary (present/absent/late counts)
POST	/api/attendance/regularize	→ employee submits regularization request
GET	/api/attendance/regularize	→ list regularization requests [HR_ADMIN]
PATCH	/api/attendance/regularize/:id	→ approve/reject [HR_ADMIN]

Business rules:

- Only one punch-in per employee per day
- Compute `workHours` on punch-out: $(\text{punchOutTime} - \text{punchInTime})$ in decimal hours
- status auto-computed: check punch-in time vs. shift start (grace period configurable)
 - Before shift start + grace: PRESENT
 - Within late tolerance (e.g., 30 min after shift start): LATE
 - No punch: ABSENT (auto-marked after day ends via cron job)
 - Half day: < 4 hours work
- Geofencing (optional): if `workSite` has lat/lng + radius, verify punch-in coordinates within radius
- Cron job runs at 11:59 PM to mark absent employees, compute daily summaries

3B. Backend — Leave Module

Endpoints:

GET	/api/leave/types	→ all leave types
POST	/api/leave/types	→ create type [HR_ADMIN]
PATCH	/api/leave/types/:id	→ update type [HR_ADMIN]
GET	/api/leave/balances	→ all employees' balances [HR_ADMIN]
GET	/api/leave/balances/self	→ own balances [EMPLOYEE]
POST	/api/leave/balances/credit	→ manual credit [HR_ADMIN]
GET	/api/leave/requests	→ all requests [HR_ADMIN]; own [EMPLOYEE]
POST	/api/leave/requests	→ apply for leave [EMPLOYEE]
PATCH	/api/leave/requests/:id	→ approve/reject [HR_ADMIN]; cancel [EMPLOYEE if

Leave apply logic:

1. Validate: no overlapping leave requests, sufficient balance, advance notice check
2. Compute `totalDays` (excluding weekends and holidays)
3. Create `LeaveRequest` (status: PENDING)
4. Deduct from `pendingDays` in `LeaveBalance`

5. Send notification to HR Admin

6. On approval: move pendingDays → usedDays, mark attendance records as ON_LEAVE

7. On rejection: restore balance, notify employee

3C. Frontend — Attendance (Employee)

Punch In/Out Widget on Dashboard:

- Large card showing: today's date, current time (live), shift details
- Big "Punch In" button (teal) → calls GPS API → sends to backend
- After punch-in: shows time punched in, live elapsed work hours counter
- "Punch Out" button appears after punch-in
- Status badge: Present / Late / On Leave / Holiday

Attendance Calendar Page:

- Monthly calendar grid
- Each day colored: Green (Present), Yellow (Late), Red (Absent), Blue (Leave), Gray (Weekend/Holiday)
- Click a day: shows punch in/out times, work hours, location
- "Regularize" button on absent days with "no reason" → opens form

3D. Frontend — Attendance (HR Admin)

Summary Dashboard Widget:

- Today's snapshot: X Present, X Absent, X Late, X On Leave out of total headcount
- Click any count → opens filtered employee list

Full Attendance Management Page:

- Employee selector (search/dropdown)
- Date range picker
- Calendar view or table view toggle
- Bulk: export attendance to Excel for selected month/range
- Regularization Requests tab: table of pending requests with approve/reject actions

3E. Frontend — Leave (Employee)

Leave Dashboard:

- Balance cards: one card per leave type showing total/used/pending/remaining
- Progress bar on each card

Apply Leave Form:

- Leave type dropdown, date picker (range), reason text area, file upload (for medical cert)
- Preview: "This will use X days. Remaining balance: Y days."

- Submit → pending status → success message

Leave History Table:

- All past requests: type, dates, days, status (badge), reason, approve/reject remarks

3F. Frontend — Leave (HR Admin)

Leave Requests Queue:

- Tabs: Pending, Approved, Rejected, Cancelled
- Table with employee name, leave type, dates, days, applied date, reason
- Quick approve/reject buttons in table row
- Click row: open drawer with full details + approve/reject with remarks

Leave Configuration:

- Leave types management: add/edit/delete leave types
- Holiday calendar: add company holidays for the year (public + restricted)
- Leave balance management: view all employees' balances, manual credit/debit

✓ PHASE 4 — PAYROLL + EXPENSES (Backend + Frontend)

Goal: Full payroll processing pipeline and expense management.

4A. Backend — Payroll Module

Endpoints:

GET	/api/payroll/structures	→ list structures [HR_ADMIN]
POST	/api/payroll/structures	→ create structure [HR_ADMIN]
PATCH	/api/payroll/structures/:id	→ update [HR_ADMIN]
POST	/api/payroll/assign-structure	→ assign structure to employee [HR_ADMIN]
POST	/api/payroll/run	→ generate payroll for month [HR_ADMIN]
GET	/api/payroll/runs	→ list payroll runs [HR_ADMIN]
GET	/api/payroll/runs/:id	→ payroll run details + all payslips [HR_ADMIN]
POST	/api/payroll/runs/:id/lock	→ lock payroll (no edits after this) [HR_ADMIN]
GET	/api/payroll/runs/:id/export	→ export payroll summary to CSV [HR_ADMIN]
GET	/api/payroll/payslips/self	→ own payslips list [EMPLOYEE]
GET	/api/payroll/payslips/:id	→ single payslip [HR_ADMIN or owner]
GET	/api/payroll/payslips/:id/pdf	→ download payslip as PDF [HR_ADMIN or owner]

Payroll Run Algorithm:

1. For each active employee in the given month:
 - a. Get their SalaryStructure and components
 - b. Get attendance for the month: presentDays, leaveDays (paid), lwpDays (LWP/unpaid)
 - c. Compute $\text{perDayBasic} = \text{basicSalary} / \text{workingDaysInMonth}$

- d. Compute `basicPayable = perDayBasic * (presentDays + paidLeaveDays)`
- e. Compute all earnings based on their `calculationType` against `basicPayable`
- f. Compute all deductions (PF, PT, TDS, LWP deduction)
- g. `grossEarnings = sum of earnings`
- h. `totalDeductions = sum of deductions`
- i. `netPayable = grossEarnings - totalDeductions`
- j. Save as Payslip

2. Save as PayrollRun with status DRAFT

PDF Generation: Use `pdfkit` or `puppeteer` to generate payslip PDF with:

- Company logo + name (Nirvigh Advisors)
- Employee details: name, code, designation, department, PAN (masked)
- Month/Year
- Earnings table (component name | amount)
- Deductions table
- Gross, Total Deductions, Net Payable
- Days: working / present / paid leave / LWP

4B. Frontend — Payroll (HR Admin)

Salary Structures Page:

- List of structures with their components
- "New Structure" button → drawer form
- Component builder: add rows with name, type (Earning/Deduction), calc type (Fixed/% of Basic/% of Gross), value
- Reorder components by drag

Payroll Run Page:

- Month/Year selector
- "Generate Payroll" button → confirmation dialog showing month and employee count
- Progress indicator while computing
- Results table: employee name, gross, deductions, net pay, status
- Individual payslip review: click row → expand/open drawer with full payslip details
- Adjust individual payslip: add manual additions/deductions
- "Lock Payroll" button (confirms: no further edits)
- "Export CSV" + "Generate PDFs for All"

4C. Frontend — Payroll (Employee)

Payslips List:

- Cards/rows per month showing: Month Year, Net Pay, Status (Generated / Locked)

- "Download PDF" button on each
- Click to expand: show earnings table + deductions table inline (no PDF needed to preview)

4D. Backend — Expenses Module

Endpoints:

POST	/api/expenses	→ submit claim [EMPLOYEE]
GET	/api/expenses	→ all claims [HR_ADMIN]; own [EMPLOYEE]
GET	/api/expenses/:id	→ single claim
PATCH	/api/expenses/:id	→ approve/reject [HR_ADMIN]
PATCH	/api/expenses/:id/payment	→ mark as paid [HR_ADMIN]
GET	/api/expenses/export	→ CSV export [HR_ADMIN]

4E. Frontend — Expenses

Submit Claim (Employee):

- Title, category dropdown, date
- Add expense items: description, amount, date, receipt upload (drag-drop)
- Running total shown
- Submit → pending

Claims List (Employee): table with title, category, amount, status badge, date submitted

Expense Approvals (HR Admin):

- Pending tab with approve/reject inline actions
- All tab with full history, filters, CSV export
- "Mark as Paid" action after approval

✓ PHASE 5 — PERFORMANCE + DOCUMENTS + REPORTS (Backend + Frontend)

Goal: Performance reviews, document management, and HR reporting.

5A. Performance Reviews

Backend endpoints:

POST	/api/performance/cycles	→ create review cycle [HR_ADMIN]
GET	/api/performance/cycles	→ list cycles
PATCH	/api/performance/cycles/:id	→ update cycle status
POST	/api/performance/goals	→ create goal [HR_ADMIN]
GET	/api/performance/goals	→ list goals for cycle+employee
PATCH	/api/performance/goals/:id	→ update progress [EMPLOYEE]

```
POST    /api/performance/reviews      → initiate review [HR_ADMIN]
PATCH  /api/performance/reviews/:id/self → employee submits self-review
PATCH  /api/performance/reviews/:id/hr   → HR submits review + final rating
```

Employee Performance Page:

- Active goals list with progress bars (employee can update %)
- Self-review form (when cycle is active): self-rating + comments for each goal
- Past reviews section: accordion or cards showing previous cycles, final ratings, HR comments

HR Admin Performance Page:

- Cycle management: create, set dates, activate/complete
- Review progress overview: X / Y employees completed self-review
- Review forms per employee: see self-review → add HR rating + comments → submit final review

5B. Document Management

Employee Documents (HR Admin):

- Per employee: list of uploaded documents with type badge, uploaded date, actions (view, download, delete)
- Upload form: document type dropdown, file upload
- Expiry date field for docs like ID proof that expire

Employee Documents (Employee — Self):

- View own documents: can download, can upload new versions for certain types (bank details, address proof)

Policy Documents (HR Admin):

- Upload company policies (PDF/DOCX): title, description, version
- Toggle active/inactive (employees only see active policies)
- List with download links

Policy Documents (Employee):

- Browse and download active company policies
- Prompted to acknowledge new policies (optional: acknowledgement checkbox)

5C. Reports & Analytics

HR Admin Reports Page (tabs):

Headcount Report:

- Total employees by department (bar chart), by employment type (donut), by work site (bar)

- New joiners by month (line chart, last 12 months)
- Attrition/exits by month

Attendance Report:

- Monthly attendance % by department (grouped bar chart)
- Late comings trend (line chart)
- Absent-heavy employees list
- Export: per-employee attendance for any month to Excel

Leave Report:

- Leave utilization by type (stacked bar)
- Employees with most leaves (table)
- Pending balance summary

Payroll Report:

- Monthly payroll cost trend (line chart)
- Salary breakdown by department (stacked bar: earnings vs. deductions)
- MIS export: full payroll sheet for accountant

Expense Report:

- Total expenses by category (donut)
- Monthly trend
- Export filtered expense claims

All charts use Recharts. All tables have CSV/Excel export using `xlsx` library.

✓ PHASE 6 — NOTIFICATIONS + SETTINGS + PRODUCTION HARDENING

Goal: Complete the system with notifications, settings, and production-readiness.

6A. Notifications System

Backend:

- `createNotification(userId, { title, message, type, linkUrl })` utility function
- Called on key events: leave approved/rejected, payslip generated, expense approved, onboarding reminder
- `GET /api/notifications` → paginated list, sorted by date
- `PATCH /api/notifications/:id/read` → mark read
- `PATCH /api/notifications/read-all`
- Optional: WebSocket ([Socket.io](https://socket.io/)) for real-time bell icon update

Frontend:

- Bell icon in TopBar with red badge showing unread count
- Click: dropdown showing last 10 notifications (type icon + title + time ago)
- "View all" → full notifications page

6B. Settings (HR Admin)

Settings Page (tabbed):

- *Company Settings*: Company name, logo upload, timezone, working week (Mon-Sat or Mon-Fri), working hours per day
- *Shift Settings*: Define default shift start/end time, grace period (minutes), late threshold
- *Leave Settings*: Leave year start month, default leave type configs
- *Holiday Calendar*: Add/edit/delete holidays for current + next year (national + company specific)
- *Work Sites*: Add/edit work site with GPS coordinates and geofencing radius
- *Departments*: Add/edit/delete departments
- *Email Settings*: Test email configuration

6C. Production Hardening

Security:

- Rate limiting: 10 requests/minute on auth endpoints (express-rate-limit + Redis store)
- Helmet.js: set secure HTTP headers (CSP, HSTS, X-Frame-Options, etc.)
- CORS: whitelist only the frontend domain
- Input sanitization: DOMPurify on frontend, Zod on backend for all inputs
- File upload validation: check MIME type + file size (5MB max for docs, 2MB for images)
- Sensitive field masking: PAN shows only last 4 chars on frontend, full value only in payslip PDF
- AuditLog on all critical backend actions

Error handling:

- Global error middleware on Express: returns structured `{ error: { code, message, details } }`
- Frontend: React Error Boundary for component crashes, global Axios error interceptor for API errors
- 404 page, 403 page, 500 page with proper messaging

Logging:

- Winston: `info` for business events, `warn` for suspicious activity, `error` for exceptions
- Log to file (daily rotation) + console in development
- Production: ship logs to Datadog or Logtail

Performance:

- Pagination on all list endpoints (default 20 per page, max 100)
- DB indexes on: `employee.email`, `employee.employeeCode`, `attendanceRecord(employeeId, date)`, `leaveRequest.status`, `payslip(employeeId, month, year)`
- Redis cache: company settings, leave types, departments, holiday calendar (TTL: 1 hour)
- Image optimization: compress profile photos to WebP on upload

6D. Deployment Setup

Create `docker-compose.yml`:

```
version: '3.8'
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: nirvigh_hrms
      POSTGRES_USER: hrms_user
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports
      - "5432:5432"

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://hrms_user:${DB_PASSWORD}@postgres:5432/nirvigh_hrms
      REDIS_URL: redis://redis:6379
      JWT_SECRET: ${JWT_SECRET}
      JWT_EXPIRY: 8h
      SMTP_HOST: ${SMTP_HOST}
      SMTP_USER: ${SMTP_USER}
      SMTP_PASS: ${SMTP_PASS}
      SMTP_FROM: "Nirvigh HRMS <noreply@nirvighadvisors.com>"
      APP_URL: https://hrms.nirvighadvisors.com
      NODE_ENV: production
    depends_on:
      - postgres
      - redis
    ports:
      - "4000:4000"

  volumes:
    pgdata:
```

`.env.example` file (include this):

```
DATABASE_URL=
REDIS_URL=
JWT_SECRET=<generate 64-char random string>
JWT_EXPIRY=8h
SMTP_HOST=
SMTP_PORT=587
SMTP_USER=
SMTP_PASS=
SMTP_FROM=noreply@nirvighadvisors.com
APP_URL=https://hrms.nirvighadvisors.com
AWS_S3_BUCKET=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_REGION=ap-south-1
NODE_ENV=production
```

☒ EMAIL TEMPLATES (Nodemailer)

All emails must be proper HTML templates, not plain text. Build these:

1. Invite Email:

Subject: Welcome to Nirvigh Advisors HRMS – Activate Your Account

Content: Logo, greeting with employee name, description, big "Activate Account" button with token link, note that link expires in 72 hours, footer with company details.

2. Leave Approval Email (to Employee):

Subject: Your Leave Request has been [Approved/Rejected]

Content: Leave details table (type, dates, days), status badge (green/red), HR remarks if any, leave balance summary.

3. Payslip Generated Email (to Employee):

Subject: Your Payslip for [Month Year] is ready

Content: Month, net payable amount (large, prominent), "View Payslip" button linking to HRMS, note to login.

4. Password Reset Email:

Subject: Reset Your Nirvigh HRMS Password

Content: Reset button with link, 1-hour expiry notice.

☒ KEY UI/UX DETAILS

Sidebar Structure (HR Admin):

[Nirvigh Advisors Logo]

Dashboard

— PEOPLE —

Employees

— OPERATIONS —

```
Attendance
Leave
— FINANCE —
Payroll
Expenses
— DEVELOPMENT —
Performance
— RESOURCES —
Documents & Policies
— INSIGHTS —
Reports
— CONFIG —
Settings
_____
[User Avatar] Vikram Desai
[Logout icon]
```

Sidebar Structure (Employee):

```
[Nirvigh Advisors Logo]
_____
Dashboard
My Attendance
My Leaves
My Payslips
My Expenses
My Performance
My Documents
Notifications
_____
[User Avatar] [Name]
[Logout icon]
```

Dashboard Widgets Layout (HR Admin) — CSS Grid:

```
Row 1: [Headcount KPI] [Present Today KPI] [Pending Leaves KPI] [Payroll Status KPI]
Row 2: [Attendance Today – donut chart + list] [Leave Requests – queue table]
Row 3: [Joiners/Exits trend – line chart] [Department headcount – bar chart]
Row 4: [Announcements] [Upcoming events / birthdays / work anniversaries]
```

Dashboard Widgets Layout (Employee) — CSS Grid:

```
Row 1: [Punch In/Out Widget – large] [Leave Balance summary – 3 mini cards]
Row 2: [Attendance Calendar – mini monthly view] [Latest Payslip card]
Row 3: [Pending Tasks – e.g., incomplete profile, pending self-review] [Company Not
```

Color coding for attendance statuses (use consistently everywhere):

- Present: `--color-success (#437a22)`
- Late: `--color-warning (#964219)`

- Absent: `--color-error (#a12c7b)`
- On Leave: `--color-blue (#006494)`
- Holiday: `--color-text-muted`
- Weekend: `--color-surface-offset`

Employee Status Badge colors:

- Active: `success`
- Inactive: `neutral/muted`
- Terminated: `error`
- On Leave: `blue`
- Invited (pending activation): `warning/gold`

☒ COMPLETE EMPLOYEE JOURNEY (End-to-End Flow)

How It Works From Day One:

1. HR Admin logs in to HRMS
↓
2. HR Admin goes to Employees → Add Employee
Fills: Name, Work Email, Designation, Department, Site, Joining Date
↓
3. System creates Employee + User record (status: INVITED)
Generates secure invite token → sends email to work email
↓
4. Employee receives email: "Welcome to Nirvigh Advisors HRMS"
Clicks "Activate Account" link
↓
5. Employee lands on `/activate?token=xxx`
Sees "Set Your Password" form
Sets password → account activated
↓
6. Employee auto-logged in → sees Onboarding Wizard
Fills personal info, address, emergency contacts
Uploads documents (PAN, Aadhaar, bank, photo)
↓
7. Employee lands on Dashboard for the first time
Sees Punch In widget → Punches In (GPS captured)
↓
8. During the day: employee can apply leave, view attendance, raise expense claims, view payslips
↓
9. At end of month: HR Admin runs payroll
Payslip generated → employee gets email → views in HRMS
↓
10. Twice a year: HR Admin creates Review Cycle
Employee fills self-review → HR Admin reviews → final rating set
↓

11. When employee leaves: HR Admin marks TERMINATED
Account deactivated, data retained for records

☒ WHAT TO BUILD FIRST (Priority Order)

Build in exactly this order:

1. Database schema (Prisma migrations)
2. Backend auth module (login, activate, JWT)
3. Frontend auth pages (login, activate, route guards)
4. Design system tokens + base components
5. App shell (sidebar, topbar, layouts)
6. Employee CRUD + invite flow (backend + frontend)
7. HR Admin employee list + profile pages
8. Employee onboarding wizard
9. Attendance (punch in/out, calendar, HR view)
10. Leave management (types, apply, approve, balance)
11. Payroll (structures, run, payslips, PDF)
12. Expenses (submit, approve, pay)
13. Performance (cycles, goals, reviews)
14. Documents + policies
15. Reports + charts
16. Notifications
17. Settings
18. Security hardening + audit logs
19. Docker setup + deployment config
20. Final QA: test all flows, mobile responsiveness, dark mode

✓ FINAL QUALITY CHECKLIST

Before declaring production-ready, verify:

- [] All API routes require authentication (no public endpoints except login/activate/forgot-password)
- [] Every route has RBAC check — employees cannot access any /admin routes
- [] Employees can only see/edit their OWN data (check `employeeId === req.user.employeeId` on backend)
- [] Invite tokens are hashed in DB — raw token only in email

- ☐ Passwords hashed with bcrypt (cost factor 12)
- ☐ All forms have server-side validation (Zod schemas)
- ☐ File uploads: MIME type checked, size limited, stored in S3 (not local in prod)
- ☐ Sensitive fields (PAN, bank account) masked in API responses
- ☐ All lists are paginated
- ☐ Payroll can only be run once per month (unique constraint)
- ☐ AuditLog written on: login, employee create/edit/terminate, leave approve/reject, payroll run/lock, role changes
- ☐ Rate limiting on auth endpoints
- ☐ CORS locked to production frontend URL
- ☐ Environment variables used for ALL secrets (no hardcoded values)
- ☐ Dark mode works on all pages
- ☐ Mobile responsive: sidebar collapses, tables scroll horizontally, forms stack properly
- ☐ Empty states on all data tables
- ☐ Loading skeletons on all data-fetching views
- ☐ Error boundaries in React catch component crashes
- ☐ 403 and 404 pages exist
- ☐ All emails have proper HTML templates
- ☐ DB has indexes on high-query columns
- ☐ Redis caching for settings/config data
- ☐ Docker Compose tested: bring up with `docker-compose up -d` and all services connect

End of Nirvigh Advisors HRMS Master Development Prompt

Company: Nirvigh Advisors / System: HRMS / Version: 1.0 / Date: May 2026